SecureScope

WEBSITE SECURITY REPORT

example.com

Passive checks of the public address https://example.com/, scanned 23 October 2026, 12:00 UTC.

Prepared by Example Agency (sample)

82/100
Grade B

SAMPLE REPORT · a demonstration on example.com, a domain reserved for documentation. Not a real customer or scan.

OVERALL RISK RATING

Medium

Some weaknesses were found. They are worth fixing, but they are not an emergency.

Security score 82/100 (grade B). 82/100: 3 scored finding(s) account for 18 point(s) of the deduction, and 1 informational note(s) take off none. The score is a rough guide, not a measurement of security.

0	0	1	2	1
CRITICAL	HIGH	MEDIUM	LOW	INFORMATION

Summary

4 findings to review.

Start with:

- Content-Security-Policy header is missing
- Cookies set without the HttpOnly attribute
- Strict-Transport-Security max-age is short

and 1 more, listed from page 2.

PREPARED BY
Example Agency (sample)

REPORT ID
SSR-SAMPLE0000

SCAN DATE
23 October 2026, 12:00 UTC

REPORT CREATED
23 October 2026, 12:00 UTC

This report comes from a limited set of passive, non-exploitative checks and a heuristic score. An empty findings list is not proof of security, and this is not a penetration test.

Findings

Most important first. Each finding says what was seen, where, and how to fix it.

1. Content-Security-Policy header is missing MEDIUM

SS-HDR-CSP-001 · Security headers · Affected URL: https://example.com/

What this means. A Content-Security-Policy limits where scripts and other resources can load from and reduces the impact of cross-site scripting.

Evidence

```
Content-Security-Policy was not present. Checked the HTTP 200
response from https://example.com/. Security headers seen: strict-
transport-security: max-age=300; x-content-type-options: nosniff;
referrer-policy: strict-origin-when-cross-origin; permissions-policy:
camera=(), microphone=().
```

How to fix it. Define a CSP that restricts script sources, e.g. start in report-only mode with "default-src 'self'" and tighten from there.

Why it matters. Without a Content-Security-Policy, a cross-site scripting flaw anywhere on the page is easier to exploit because the browser has no limit on where scripts may load from.

Step 1: test without enforcing (Nginx)

```
add_header Content-Security-Policy-Report-Only "default-src 'self';
object-src 'none'; base-uri 'self'; frame-ancestors 'none'" always;
```

Step 2: enforce once nothing important is reported (Nginx)

```
add_header Content-Security-Policy "default-src 'self'; object-src
'none'; base-uri 'self'; frame-ancestors 'none'" always;
```

Apache (needs mod_headers)

```
Header always set Content-Security-Policy "default-src 'self';
object-src 'none'; base-uri 'self'; frame-ancestors 'none'"
```

How to verify the fix

1. Run curl -sI https://<your-host>/ and look for a Content-Security-Policy line.
2. Open the site with the browser developer tools console open. Look for blocked-resource messages and fix the sources they name before enforcing.
3. Run a new scan to confirm.

2. Cookies set without the HttpOnly attribute LOW

SS-COOKIE-HTTPONLY-001 · Cookies · Affected URL: https://example.com/

What this means. Cookies without HttpOnly are readable by scripts, so a cross-site scripting flaw could steal them. This matters most for session cookies.

Evidence

```
Cookie names: session
```

How to fix it. Add HttpOnly to cookies that scripts do not need to read, especially session cookies.

Why it matters. Scripts can read cookies without HttpOnly, so a cross-site scripting flaw could steal a session cookie.

Raw header

```
Set-Cookie: session=<value>; Secure; HttpOnly; SameSite=Lax; Path=/
```

Django (settings.py)

```
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
```

Express (express-session)

```
app.use(session({ cookie: { secure: true, httpOnly: true, sameSite:
'lax' } }));
```

How to verify the fix

1. Load the page, then check the Set-Cookie headers (curl -sI https://<your-host>/) or the Application tab in the browser developer tools.
2. Confirm the named cookie now has the attribute that was missing.
3. Run a new scan.

3. Strict-Transport-Security max-age is short LOW

SS-HDR-HSTS-002 · Security headers · Affected URL: https://example.com/

What this means. A short HSTS lifetime reduces how long browsers enforce HTTPS.

Evidence

```
Strict-Transport-Security: max-age=300
```

How to fix it. Use a max-age of at least 15552000 seconds (180 days); 1 year is common.

Why it matters. A short lifetime means browsers stop enforcing HTTPS sooner, which leaves a longer window for downgrade attacks.

Nginx (server or location block)

```
add_header Strict-Transport-Security "max-age=31536000;
includeSubDomains" always;
```

Apache (needs mod_headers)

```
Header always set Strict-Transport-Security "max-age=31536000;
includeSubDomains"
```

Cloudflare Pages or Netlify _headers file

```
/*
  Strict-Transport-Security: max-age=31536000; includeSubDomains
```

How to verify the fix

1. Run curl -sI https://<your-host>/ and read the Strict-Transport-Security line.
2. Check that max-age is at least 15552000 (180 days). One year (31536000) is common.
3. Run a new scan to confirm.

4. No security.txt contact file INFORMATION

SS-DISC-SEC-001 · Security hygiene · Affected URL: https://example.com/.well-known/security.txt

What this means. A security.txt file (RFC 9116) tells researchers how to report a vulnerability to you. Without one, reports may not reach the right person.

Evidence

```
GET https://example.com/.well-known/security.txt returned HTTP 404 without a valid Contact field.
```

How to fix it. Publish /.well-known/security.txt with at least a Contact line and an Expires date, for example: Contact: mailto:security@your-domain

Why it matters. Without a security contact, researchers who find a problem may not know how to reach you, or may not report it.

File: /.well-known/security.txt

```
Contact: mailto:security@<your-domain>
Expires: <a date within the next 12 months, ISO 8601>
Preferred-Languages: en
```

How to verify the fix

1. Open https://<your-host>/.well-known/security.txt. It should return the file, not an error or your home page.
2. Check that it has a Contact line and an Expires date in the future.
3. Run a new scan.

Remediation plan

Work through these in order. After you make changes, run a new scan to confirm they worked.

1. **Content-Security-Policy header is missing** (Medium). Define a CSP that restricts script sources, e.g. start in report-only mode with "default-src 'self'" and tighten from there.
2. **Cookies set without the HttpOnly attribute** (Low). Add HttpOnly to cookies that scripts do not need to read, especially session cookies.
3. **Strict-Transport-Security max-age is short** (Low). Use a max-age of at least 15552000 seconds (180 days); 1 year is common.
4. **No security.txt contact file** (Information). Publish /.well-known/security.txt with at least a Contact line and an Expires date, for example: Contact: mailto:security@your-domain

What was checked and passed

- The page was served over HTTPS
- X-Content-Type-Options is nosniff
- Referrer-Policy is set
- Permissions-Policy is set
- No software version shown in the Server or X-Powered-By headers
- No directory listing in the page
- Scan stayed on the verified site
- The page loads nothing over plain HTTP
- Scripts and styles from other sites carry integrity hashes
- No risky cross-origin (CORS) headers
- robots.txt and sitemap.xml do not point at sensitive paths
- No exposed .git directory or .env file
- TLS certificate is valid for this host and not close to expiry

How the score is worked out

- The score starts at 100. Each finding takes off points by severity (critical 40, high 20, medium 10, low 4); informational notes take off none. The result is kept between 0 and 100.
- Grades: A from 90, B from 80, C from 65, D from 50, F below 50.
- The same findings always give the same score. The score only reflects the passive checks that ran in this scan, so it measures how many of those checks reported a problem and how serious each was.
- It does not measure how secure the website is overall. It is not a penetration test, and it says nothing about code, logins, business logic, servers or anything the checks did not look at.

Limits of this report

- Passive checks only: one page load of the address you scanned, plus any same-site redirects, and the fixed requests listed below.
- Only the response headers and the start of the page were examined; other pages, forms and APIs were not.
- No login, no active testing, no vulnerability exploitation, no scan of subdomains.
- TLS: one verified connection is made to read the certificate and negotiated protocol. This is not a full TLS test: supported protocol versions, cipher suites and known TLS vulnerabilities are not scanned.
- Five fixed paths (/.well-known/security.txt, /.git/HEAD, /.env, /robots.txt, /sitemap.xml) are requested once each; no other paths are guessed or crawled.

- For a main domain, its public DNS records (SPF, DMARC, CAA and DNSSEC) are looked up. Subdomains skip these checks, and a lookup that fails is shown as not checked.

Report SSR-SAMPLE0000 · created 23 October 2026, 12:00 UTC · prepared by Example Agency (sample) · generated by SecureScope